

Geometric Transformations Using OpenCV

NAME : INIYA E

REG NO : 212224230096

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: # Step 1: Load the image
image = cv2.imread('Chennai.jpg') # Load the image from file
```

```
In [3]: # Display the original image
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for correct displ
plt.title("Original Image")
plt.axis('off')
```

Out[3]: (-0.5, 735.5, 1136.5, -0.5)

Original Image



```
In [4]: Step 2: Image Translation
tx, ty = 100, 50 # Translation factors (shift by 100 pixels horizontally and 50 vertically)
translation = np.float32([[1, 0, tx], [0, 1, ty]]) # Translation matrix:
[1, 0, tx] - Horizontal shift by tx
[0, 1, ty] - Vertical shift by ty
translated_image = cv2.warpAffine(image, M_translation, (image.shape[1], image.shape[0]))
```

```
In [5]: plt.imshow(cv2.cvtColor(translated_image, cv2.COLOR_BGR2RGB)) # Display the translated im
plt.title("Translated Image")
plt.axis('off')
```

Out[5]: (-0.5, 735.5, 1136.5, -0.5)



```
In [6]: # Step 3: Image Scaling
fx, fy = 5.0, 2.0 # Scaling factors (1.5x scaling for both width and height)
scaled_image = cv2.resize(image, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR)
# resize: Resize the image by scaling factors fx, fy
# INTER_LINEAR: Uses bilinear interpolation for resizing
```

```
In [7]: plt.imshow(cv2.cvtColor(scaled_image, cv2.COLOR_BGR2RGB)) # Display the scaled image
plt.title("Scaled Image") # Set title
plt.axis('off')
```

Out[7]: (-0.5, 3679.5, 2273.5, -0.5)

Scaled Image



```
In [8]: # Step 4: Image Shearing
shear_matrix = np.float32([[1, 0.5, 0], [0.5, 1, 0]]) # Shearing matrix
# The matrix shears the image by a factor of 0.5 in both x and y directions
# [1, 0.5, 0] - Shear along the x-axis (horizontal)
# [0.5, 1, 0] - Shear along the y-axis (vertical)
sheared_image = cv2.warpAffine(image, shear_matrix, (image.shape[1], image.shape[0]))
```

```
In [9]: plt.imshow(cv2.cvtColor(sheared_image, cv2.COLOR_BGR2RGB)) # Display the sheared image
plt.title("Sheared Image") # Set title
plt.axis('off')
```

Out[9]: (-0.5, 735.5, 1136.5, -0.5)



```
In [10]: Step 5: Image Reflection
lected_image = cv2.flip(image, 2) # Flip the image horizontally (1 means horizontal flip)
lip: 1 means horizontal flip, 0 would be vertical flip, -1 would flip both axes
```

```
In [11]: plt.imshow(cv2.cvtColor(reflected_image, cv2.COLOR_BGR2RGB)) # Display the reflected image
plt.title("Reflected Image") # Set title
plt.axis('off')
```

Out[11]: (-0.5, 735.5, 1136.5, -0.5)

Reflected Image



```
In [12]: 5: Image Rotation
, width) = image.shape[:2] # Get the image height and width
45 # Rotation angle in degrees (rotate by 45 degrees)
= (width // 2, height // 2) # Set the center of rotation to the image center
ion = cv2.getRotationMatrix2D(center, angle, 1) # Get the rotation matrix
tionMatrix2D: Takes the center of rotation, angle, and scale factor (1 means no scaling)
_image = cv2.warpAffine(image, M_rotation, (width, height)) # Apply rotation
```

```
In [13]: plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB)) # Display the rotated image
plt.title("Rotated Image") # Set title
plt.axis('off')
```

Out[13]: (-0.5, 735.5, 1136.5, -0.5)

Rotated Image



```
In [14]: # Step 7: Image Cropping
x, y, w, h = 100, 100, 200, 150 # Define the top-left corner (x, y) and the width (w) and
# Cropping the image from coordinates (x, y) to (x+w, y+h)
cropped_image = image[y:y+h, x:x+w]
# The crop is performed by slicing the image array in the y and x directions
```

```
In [15]: plt.imshow(cv2.cvtColor(cropped_image, cv2.COLOR_BGR2RGB)) # Display the cropped image  
plt.title("Cropped Image") # Set title  
plt.axis('off')
```

Out[15]: (-0.5, 199.5, 149.5, -0.5)

